WEEK 3

Classical IR Models

Boolean Model, Vector Space Model, Clustering and Classification in IR



For IR: Development

For building practical information retrieval systems

- Development of Boolean search module (AND, OR, NOT)
- Development of vector-based ranking module (term weights and similarity)

For Advanced IR: Theory

For understanding the mathematical models and techniques

- Mathematical expression of Boolean logic
- Vector representation and weighting
- Similarity measures
- Clustering models
- Classification models

Learning Outcomes

By the end of Week 3, students will be able to:

1. Explain the purpose of classical IR models.
2. Implement Boolean search using logical operators.
3. Develop vector-based document ranking.
4. Understand clustering and classification in IR.
5. Apply these models in FYP-based IR modules.
6. Mathematically express IR models for research/product development.



These classical models form the foundation of modern IR systems and research innovations.

What Are Classical IR Models?

Classical IR models are the early **mathematical** and **logical** models used to retrieve **relevant documents** from a collection.



They answer three major questions:

1



What is stored?

Documents, terms, indexes, and document representations.

2



Why is it retrieved?

Because the user query matches document terms or meanings.

3



How is it ranked?

Through Boolean matching, vector similarity, or model-based scoring.

Real-World Example

Suppose a student builds an FYP titled:
"Smart Course Material Search System"

A user searches:
"database normalization examples"



The IR system should:

1. **Search course files.**
   The system scans all available course materials, including notes, slides, assignments and PDFs.

2. **Match query terms.**
   The system identifies and matches the important terms in documents.

3. **Rank the most relevant files.**
   Documents are scored and ranked based on relevance.

4. **Return notes, slides, assignments, and PDFs.**
   The most relevant materials are returned to the user.

This can be developed using **Boolean Model** and **Vector Space Model**.

Boolean Model
Logical operators for precise searching

Vector Space Model
Term weighting and similarity for smarter ranking

Boolean Model — Basic Idea



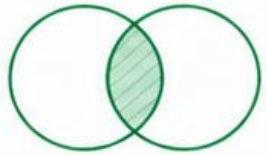
What is the Boolean Model?

The Boolean Model is a classical IR model that represents documents and queries as sets of terms and uses logical operators to retrieve documents that satisfy the query conditions.

Logical Operators

AND

AND
Both terms must be present.



A AND B

OR

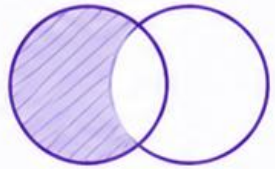
OR
At least one of the terms must be present.



A OR B

NOT

NOT
Excludes documents containing the term.



A NOT B

Boolean Model Output

It gives a clear answer:

Document is retrieved
(Matches the query condition)

Document is not retrieved
(Does not match the query)

Important Note

There is usually no ranking in the basic Boolean Model.



No ranking — only match or no match

Boolean Model — Implementation View for IR



For **FYP/product development**, Boolean search can be implemented through:

1 Text Preprocessing



Remove stopwords, punctuation, and normalize text.

2 Tokenization

Machine learning improves healthcare systems.

machine | learning | improves | healthcare | systems | ... | ...

Split text into individual terms (tokens).

3 Inverted Index Creation

Term	Doc IDs
machine	1, 2, 4
learning	1, 3, 4
healthcare	2, 4
...	...

Create mapping of terms to document IDs.

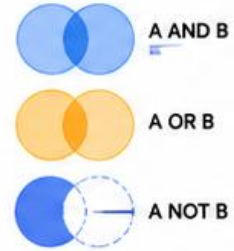
4 Query Parsing

machine learning AND healthcare

AND → machine learning | healthcare

Parse the user query and identify operators and terms.

5 AND / OR / NOT Operation



Apply Boolean operators on document sets.

6 Document Retrieval



Retrieve documents that satisfy the Boolean condition.

7 Result Display



Display matching documents to the user.

FYP Module Example:

Search only those project reports that contain both "machine learning" AND "healthcare".

machine learning + AND + healthcare = Returns only reports that contain BOTH terms.

Boolean Model — Mathematical View for Advanced IR

1. Document Representation

A document can be represented as a set of terms:

D_1 = {information, retrieval, indexing}

D_2 = {search, engine, ranking}

D_3 = {indexing, ranking, retrieval}

2. Query

$$Q = \text{indexing} \ \text{AND} \ \text{ranking}$$

3. Relevance Condition

A document is relevant if:

D contains indexing **AND** *D* contains ranking

4. Mathematical Expression

Therefore:

$$Rel(D, Q) = 1 \quad \text{if both terms exist}$$

$$Rel(D, Q) = 0 \quad \text{otherwise}$$

5. Summary

This makes Boolean retrieval simple, exact, and interpretable.



Strengths and Limitations of Boolean Model

Strengths

- Easy to understand
- Easy to implement
- Good for exact search
- Useful in legal, medical, and database-style retrieval

Limitations

- No ranking in basic form
- Too strict for natural language queries
- User must know correct operators
- Partial relevance is difficult to handle

Vector Space Model — Basic Idea

Documents and queries are represented as vectors.

Each term becomes a dimension.

Documents are ranked according to similarity with the query.

Similarity is usually measured using cosine similarity.

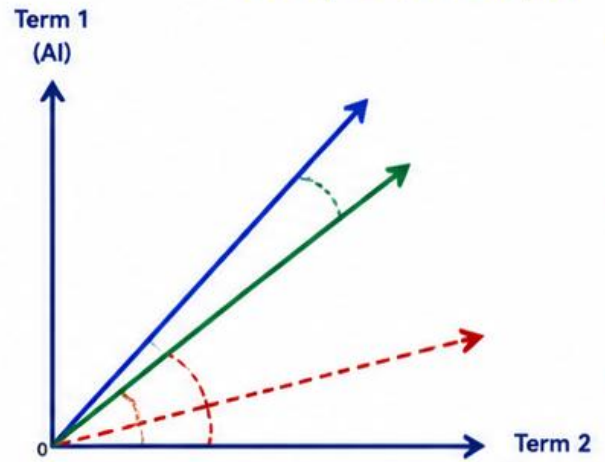
Example

Query: "AI search system"
→ The user is looking for documents about **AI search systems**.

Document A: "AI based information retrieval system"
→ Contains terms related to **AI**, **search**, and **retrieval**.

Document B: "Computer hardware installation guide"
→ Contains terms about hardware and installation, **not related to AI search**.

Vector Space (2D Example)



- **Query Vector**
  (AI search system)
  What the user is looking for.
- **Document A Vector**
  (AI based information retrieval system)
  Closer to the query → Higher similarity → Ranked higher.
- **Document B Vector**
  (Computer hardware installation guide)
  Farther from the query → Lower similarity → Ranked lower.

Document A will be **ranked higher** because it is closer (more similar) to the query vector.

Term Weighting in Vector Space Model

In VSM, all words do not have equal importance.

Common words may have low weight.

Important words may have high weight.

Important Weighting Ideas

1 Term Frequency (TF)

Measures how often a term appears in a document.

search → 3 times in this document

$$TF(t, d) = \frac{\text{Number of times term t appears in document d}}{}$$

2 Inverse Document Frequency (IDF)

Measures how rare or important a term is in the entire collection.

Appears in few documents → higher IDF

$$IDF(t) = \log \frac{N}{df(t)}$$

3 TF-IDF

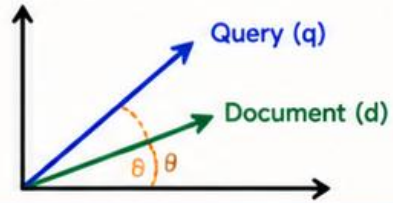
Combines TF and IDF to give higher weight to terms that are frequent in a document but rare in the collection.

TF(t, d) × IDF(t) ↓ TF-IDF(t, d)

Higher TF-IDF → More important term

4 Cosine Similarity

Measures the similarity between the query vector and document vector using the cosine of the angle between them.



$$\cos(q, d) = \frac{q \cdot d}{||q||\ ||d||}$$

N = Total number of documents df(t) = Number of documents containing term t

These weighting techniques help the model understand which terms are more important and rank documents based on their similarity to the query.

TF-IDF Concept

Term Frequency (TF)

Tells how often a term appears in a document.

Inverse Document Frequency (IDF)

Tells how rare or important a term is in the whole collection.

TF-IDF

Gives higher weight to terms that are frequent in one document but not common in all documents.

TF(t,d)
How often term t appears in document d

×

IDF(t)
How rare/important term t is in the collection

→

TF-IDF(t,d)
Final weight of term t in document d

Example

the

The word "the" appears everywhere, so it has low importance.

Document Collection

Term	Doc 1	Doc 2	Doc 3	Doc 4	Doc 5	IDF
the	25	30	22	28	31	Low
retrieval	3	0	5	2	1	High

The word "retrieval" may be more important in an IR document.

TF-IDF increases the weight of important terms and reduces the weight of common terms.

Computed Example of Vector Ranking

Query
"retrieval ranking"

Documents

D1: retrieval indexing ranking | D2: ranking search engine | D3: database normalization



1 Vocabulary (All Unique Terms)

Term	Index
retrieval	1
indexing	2
ranking	3
search	4
engine	5
database	6
normalization	7

2 Vector Representation

Order of terms → [retrieval, indexing, ranking, search, engine, database, normalization]

Query (Q) → [1, 0, 1, 0, 0, 0, 0]

D1 → [1, 1, 1, 0, 0, 0, 0]

D2 → [0, 0, 1, 1, 1, 0, 0]

D3 → [0, 0, 0, 0, 0, 1, 1]

3 Similarity with Query (Cosine Similarity)

$$sim(Q, D_1) = \frac{Q \bullet D_1}{||Q||\,||D_1||} = 0.816$$

$$sim(Q, D_2) = \frac{Q \bullet D_2}{||Q||\,||D_2||} = 0.408$$

$$sim(Q, D_3) = \frac{Q \bullet D_3}{||Q||\,||D_3||} = 0.000$$

4 Expected Ranking

1. **D1** (retrieval indexing ranking) — Score: 0.816
2. **D2** (ranking search engine) — Score: 0.408
3. **D3** (database normalization) — Score: 0.000

Explanation

- D1 has **both** retrieval and ranking
- D2 has **ranking** only
- D3 has **none** of the query terms

→ Therefore, D1 is most similar, D2 is next, and D3 is least similar.

Classification in IR

Classification assigns documents to **predefined categories**.

In an IR system, classification improves **filtering** and **retrieval**.



Example

A document can be classified as:

- Research paper
- FYP report
- Lecture notes
- Assignment
- Dataset
- Proposal

How Classification Works in IR

Input Document → Feature Extraction → Classification Model → Assigned Category

Models used:

Naïve Bayes | SVM | Decision Trees | Neural Networks



Benefits

- Better Filtering
- Improved Retrieval
- Saves Time
- Higher Accuracy
- Scalable for Large Collections

Classification helps IR systems understand the **type of content** and show more **relevant results**.

Clustering vs Classification



Feature	Clustering	Classification
Labels	Not predefined	Predefined
Learning Type	Unsupervised	Supervised
Purpose	Discover groups	Assign category
Example	Group similar FYPs	Classify FYP as AI / Cyber / SE

Clustering finds **natural groups** in data, while classification assigns items to **known categories**.

IR Development Focus

For the IR course, students must develop:



1. **Boolean search module**
   

2. Vector ranking module
   
3. Document categorization module
   
4. Simple clustering or classification support
   
5. Search interface
   

6. Search result page
   
7. Basic evaluation using precision and recall
   

These components together will help build a complete and functional Information Retrieval system.

Advanced IR Research/Product Focus

For Advanced IR, students must **extend the same models** toward:



1 Mathematical comparison

- Compare Boolean, Vector and other models
- Theoretical and empirical comparison

2 Improved ranking formula

- Propose better weighting or similarity functions
- Enhance relevance scores

3 Research gap identification

- Review existing work
- Identify open problems and limitations

4 Dataset-based evaluation

- Use standard IR datasets or build your own
- Perform experiments and analysis

5 Publishable methodology

- Design research workflow
- Ensure novelty, validity and reproducibility

6 Commercial search product

- Build a functional search system
- Focus on usability, speed and scalability

7 Funding proposal based on IR application

- Identify real-world problem
- Prepare a strong proposal for grants or industry funding

The goal is to move from **understanding** models to **advancing research** and **creating impact**.

Week 3 Case Study Assignment



Goal:
Apply IR models practically and build the foundation for research and real-world applications.

For IR Students:

Develop a mini search engine that supports **Boolean search** and **vector-based ranking**.

Required Features:

1. Upload or store at least **20 documents**.
2. Build an **inverted index**.
3. Support **AND, OR, NOT** queries.
4. Compute **vector similarity**.
5. Display **ranked results**.

For Advanced IR Students:

Write a short technical note comparing **Boolean Model** and **VSM** mathematically and propose **one improvement** for a research or commercial IR product.

Your technical note should include:

1. $f(x)$ Mathematical formulation of Boolean Model.
2. Mathematical formulation of Vector Space Model.
3. Comparison of both models (similarities and differences).
4. Identification of limitations / research gap.
5. Proposed improvement with justification and expected impact.

This assignment strengthens your **understanding**, **practical skills**, and **research mindset** in Information Retrieval.

Quiz

Answer the following questions based on what you learned in Week 3.

1. Why does the Boolean Model not naturally support ranking?
2. What is the role of TF-IDF in VSM?
3. How does cosine similarity help in ranking?
4. What is the difference between clustering and classification?
5. Which model is more suitable for FYP product search: Boolean or VSM? Justify.

Summary

Week 3 covered:

- Boolean Model
- Vector Space Model
- TF-IDF
- Cosine Similarity
- Clustering
- Classification
- FYP-based IR module development
- Mathematical and research-oriented interpretation for Advanced IR



These concepts form the **foundation** for building and researching effective **Information Retrieval** systems.